

WRITING AND UNDERSTANDING DOCKERFILES

❖ Introduction to Dockerfile

- A **Dockerfile** is a plain text script containing instructions used by Docker **to automate the creation of container images**.
- Each instruction in the Dockerfile builds a layer in the final image, making the image easy to reuse and cache.

❖ Basic Dockerfile Instructions

Instruction	Description	Example
FROM	Sets the base image for your container. Must be the first instruction.	FROM ubuntu:20.04 [OR] FROM python:3.9-alpine
LABEL	Adds metadata to the image (like version, author, description)	LABEL version="1.0" LABEL description="My web application"
RUN	Executes commands during image build	RUN apt-get update
CMD	Provides default command when container starts	CMD ["python", "app.py"] CMD ["nginx", "-g", "daemon off;"]
ENTRYPOINT	Sets the main command that always runs. Cannot be overridden	ENTRYPOINT ["python", "app.py"]
ENV	Sets environment variables inside the container.	ENV NODE_ENV=production ENV PATH="/app/bin:\${PATH}"
ARG	Defines build-time variables (available only during image build).	ARG VERSION=latest ARG BUILD_DATE
COPY	Copies files/directories from host to container (relative paths).	COPY app.py /usr/src/app/ COPY . /app
ADD	Similar to COPY but with extra features (auto-extracts tar files, supports URLs)	ADD https://example.com/file.tar.gz /tmp/
EXPOSE	Documents which ports the container uses	EXPOSE 80
USER	Sets the user the container should run as.	USER nobody USER 1001
WORKDIR	Sets the working directory inside the container for commands.	WORKDIR /app WORKDIR /usr/src/app
VOLUME	Creates mount points for external volumes	VOLUME ["/data"] VOLUME /var/log

❖ Common Dockerfile Example

```
FROM ubuntu:20.04
LABEL maintainer="trainer@example.com"
RUN apt-get update && apt-get install -y nginx
COPY index.html /var/www/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

▪ This **Dockerfile**:

1. Uses Ubuntu as the base image
2. Installs NGINX
3. Copies an HTML file
4. Exposes port 80
5. Starts NGINX on container startup

▪ **Docker Image Build Process**

- To build an image using a Dockerfile:
- ***docker build . -t image:tag***
- 1. -t sets the image name and optionally a tag

❖ Docker Image Push and Pull

- **Login to DockerHub**
- **docker login -u user_name**

➤ **Push to DockerHub:**

- **docker push username/image:tag**

➤ **Pull an Image from DockerHub**

- **docker pull username/image:tag**

❖ **Best Practices**

- Use small **base images like alpine** when possible.
- **Combine RUN commands** to reduce image layers.
- Use **.dockerignore** to exclude unnecessary files from context.
- Use **multi-stage builds** to separate build and runtime environments

[GENERAL INTERVIEW QUESTIONS]

Q1. What is a Dockerfile?

- A Dockerfile is a script with a set of instructions used to build a Docker image automatically.

Q2. What is the purpose of the FROM instruction in a Dockerfile?

- FROM defines the base image that the new image will be built on. It must be the first instruction.

Q3. What is the difference between CMD and ENTRYPOINT?

- CMD provides default arguments that can be overridden. ENTRYPOINT defines the main executable and is less easily overridden

Q4. What does the RUN instruction do?

Q5. What's the use of the COPY command in Dockerfile?

- **COPY transfers** files/directories from the host machine into the image.

Q6. How is ADD different from COPY?

- ADD can copy files, handle remote URLs, and automatically extract archives.
COPY is simpler and preferred for local files.

Q7. What does the EXPOSE instruction do?

- It documents the port the container will listen on. It does **not** publish the port automatically

Q8. How do you build a Docker image from a Dockerfile?

- Command Using: `docker build -t image-name .`

Q9. How can you push a Docker image to DockerHub?

1. Login with `docker login`
2. Tag the image `Push`
3. using `docker push username/image:tag`

Q10. What is the purpose of a Dockerfile?

- A Dockerfile automates the process of creating a Docker image using a list of instructions.

Q11. What is a multi-stage build in Docker?

- It involves using multiple **FROM** statements to separate build-time and runtime environments to reduce image size.

Q12. How does a frontend Dockerfile differ from a backend one?

- **Frontend** often uses multi-stage builds (e.g., build with Node.js, serve with NGINX), while **backend** usually runs a server like Node.js or Flask directly.

Q13. What is the command to run a MySQL container with environment variables?

- `docker run -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb mysql:8`

Q14. How can you check logs of a running Docker container?

- `docker logs container-name`

Q15. How would you troubleshoot connectivity between containers?

- Check:
 1. If containers are on the same network (`docker network inspect`)
 2. If ports are correctly exposed
 3. Use `ping` or `curl` from one container to another using container names

Q16. What is Docker Compose?

- A tool for defining and managing multi-container Docker applications using a YAML file.

Q17. What is the purpose of docker-compose.yml?

- It describes the services, networks, volumes, and configurations required to run a multi-container application.

Q18. How do you start services defined in a docker-compose.yml file?

- *Using the command:-* `docker-compose up`

Q19. What is the difference between docker-compose up and docker-compose up -- build?

- **up** starts containers using existing images.
- **up --build** forces Docker to rebuild the images before starting

Q20. How do you stop and remove all services started by Docker Compose?

- `docker-compose down`

Q21. What happens when you run docker-compose up?

- Docker builds the images (if needed), creates the network, and starts all the containers defined in the Compose file.

Q22. How do containers communicate with each other in Docker Compose?

- They **communicate** over an automatically created default network using their **service** names as **hostnames**.

Q23. How do you map ports using Docker Compose?

- Under the ports key: `ports:- "8080:80"`